

ŽILINSKÁ UNIVERZITA V ŽILINE
Fakulta riadenia
a informatiky

Semestrálna práca z predmetu Jazyk C# a .NET

DISEASE SIMULATION

Vypracoval: Emma Lujza Rothová

Predmet: Jazyk C# a .NET

Študijná skupina: 5ZYI22

Cvičiaci: Ing. Štefan Toth, PhD

Termín cvičenia: štvrtok 12:00

V Žiline 19.5. 2026

Obsah

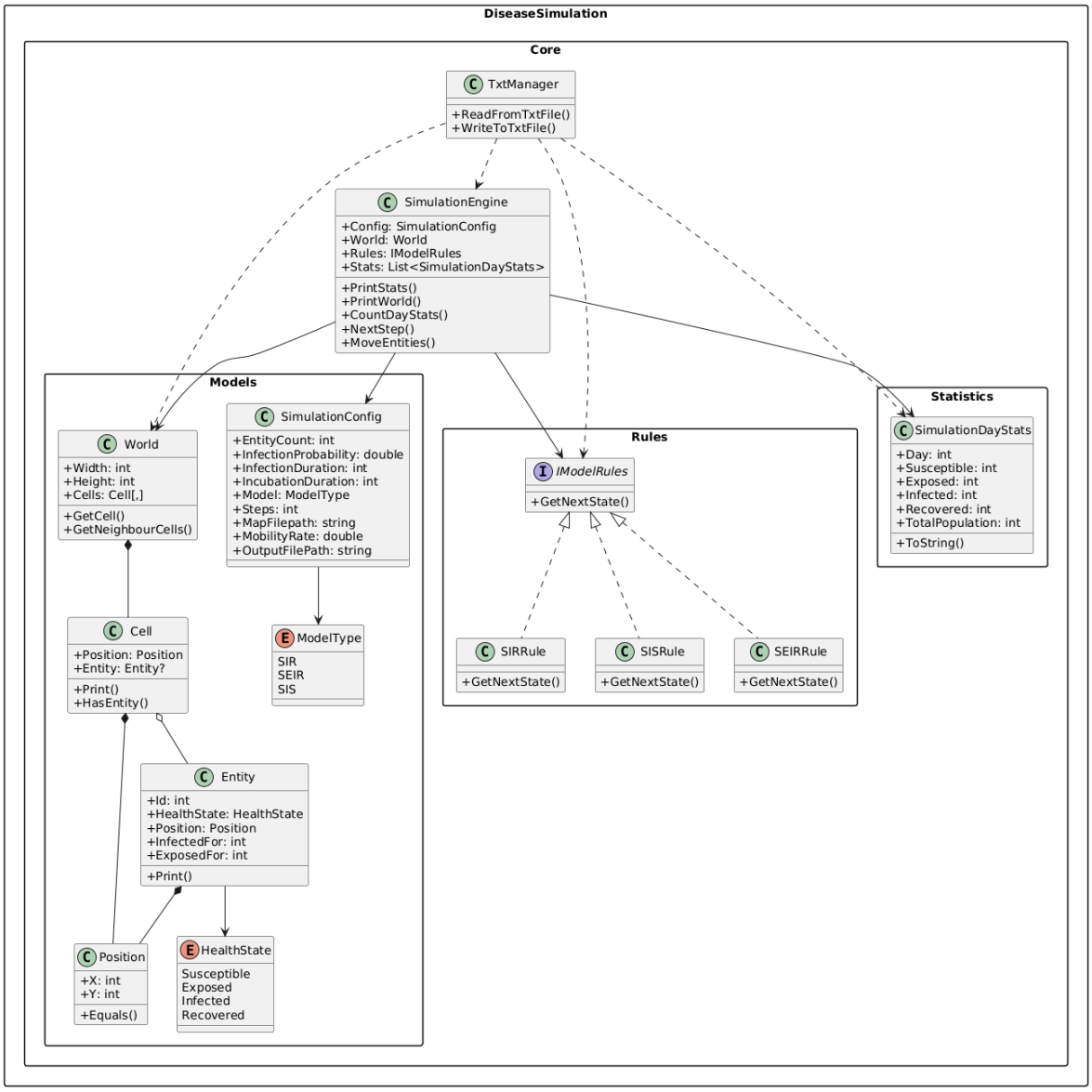
1. Popis programu 3
2. Programátorská príručka 4
2.1 Diagram tried 4
2.2 Zoznam použitých technológií 6
3. Používateľská príručka 8
3.1 Konzolová aplikácia 8
3.2 Grafická aplikácia 8
4. Prehľad problémov 11
5. Záver 12
5.1 Zhodnotenie implementácie 12
5.2 Návrh na ďalší rozvoj 12

1. Popis programu

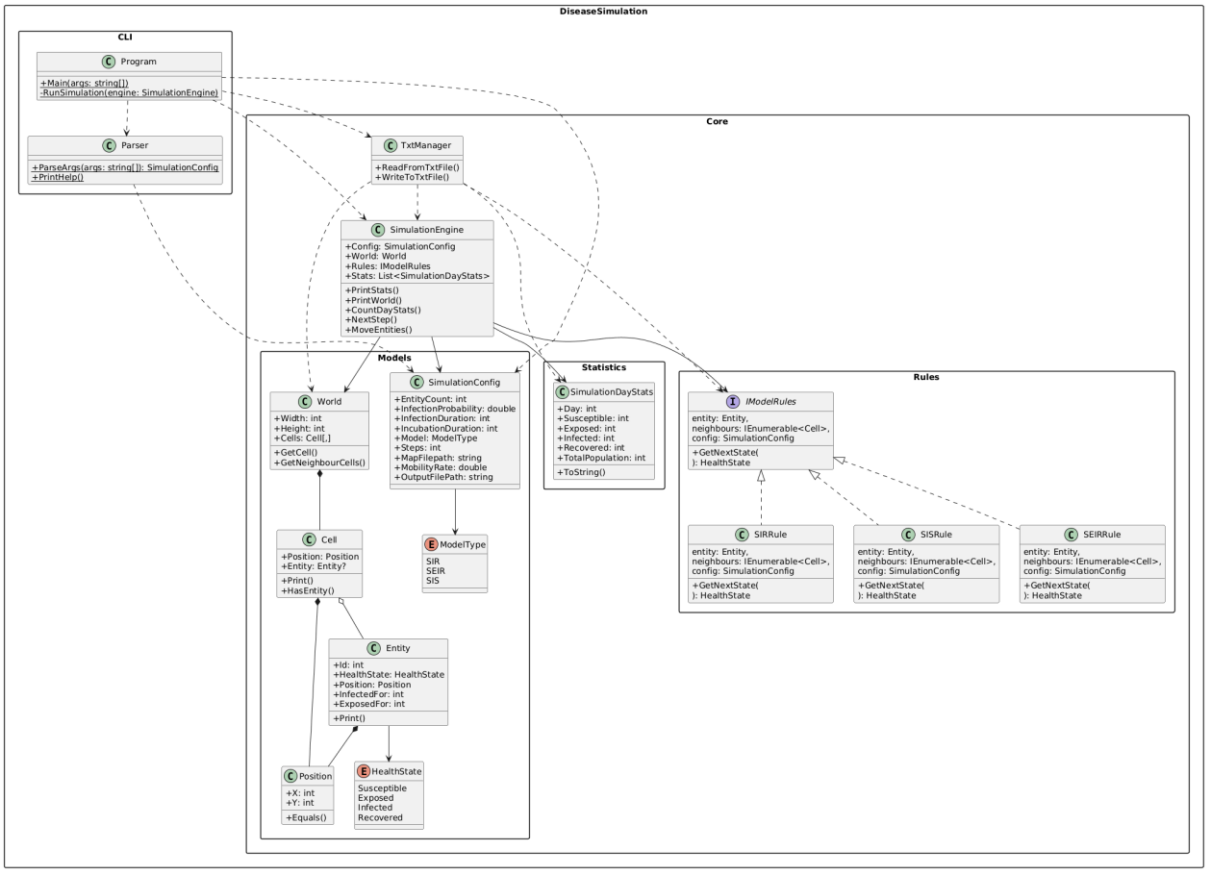
Projekt Disease Simulation je program, ktorý slúži na simuláciu šírenia choroby na určitej vzorke obyvateľstva nejakého územia. Používateľ má na výber z troch typov simulácii (SIS, SIR, SEIR). Aplikácie zbiera za každý deň šírenia nákazy štatistiky o počte zdravých ľudí, ľudí v inkubačnej dobe, infikovaných ľudí a ľudí, ktorí chorobu prekonali a majú vybudovanú imunitu.

2. Programátorská príručka

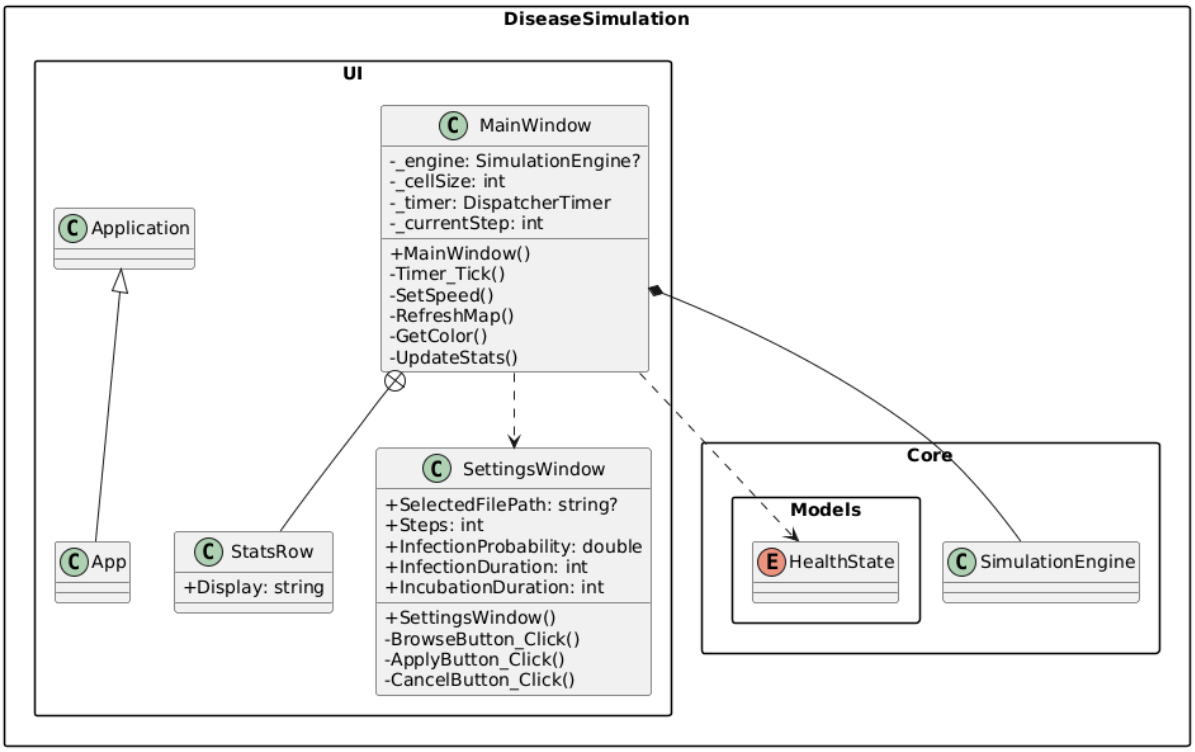
2.1 Diagram tried



Obrázok 1: Diagram tried knižnice programu



Obrázok 2: Diagram tried CLI časti programu



Obrázok 3: Diagram tried UI časti programu

2.2 Zoznam použitých technológií

V tejto kapitole sú popísané kľúčové technológie, programovacie jazyky, vývojové prostredia a knižnice, ktoré boli využité pri návrhu, implementácii a testovaní simulačného prostredia pre modelovanie šírenia chorôb.

Vývojová platforma a programovací jazyk:

Celá aplikačná logika vrátane simulačného jadra, konzolového rozhrania a grafického prostredia je implementovaná v objektovo orientovanom jazyku C#. Platforma .NET 10 bola zvolená kvôli jej najmodernejšej optimalizácii, vylepšenému behovému prostrediu (Runtime), vysokej výkonnosti pri práci s maticovými dátami a natívnej cross-platformovej podpore pre core logiku.

Konzolové rozhranie:

Pre konzolové rozhranie bola použitá technológia .NET Console Application. Využíva štandardné vstupné a výstupné prúdy (System.Console) a prispôsobený algoritmus na parsovanie komplexných argumentov (napr. --map, --steps, --infProb ...), čo umožňuje jednoduchú integráciu simulátora do externých skriptov.

Grafické používateľské rozhranie:

Pre implementáciu desktopovej aplikácie bola využitá technológia WPF (Windows Presentation Foundation). Umožňuje striktné oddelenie vzhľadu aplikácie od aplikačnej logiky. V projekte zabezpečuje interaktívne ovládanie simulácie v reálnom čase.

V rámci tejto vrstvy boli použité technológie:

- **XAML (Extensible Application Markup Language)**: Deklaratívny jazyk použitý na definíciu používateľského rozhrania, rozloženia prvkov (Grid, StackPanel, DockPanel) a dizajnu okien MainWindow a SettingsWindow.
- **DispatcherTimer**: Špeciálny časovač integrovaný s UI vláknom WPF, ktorý riadi krokovanie simulácie (NextStep) a v pravidelných intervaloch spúšťa aktualizáciu mapy a prepočet štatistík bez mrznutia rozhrania.
- **WPF Drawing & Shapes**: Prvky ako Rectangle a Canvas boli využité na dynamické renderovanie mriežky sveta a vizualizáciu epidemiologického stavu entít pomocou farebného kódovania.

Perzistencia dát a I/O operácie:

Aplikácia nevyžaduje databázový systém, dáta spracováva lokálne pomocou textových súborov, čo uľahčuje prenosnosť a konfiguráciu:

- **StreamReader / StreamWriter**: Využívané v triede TxtManager na sekvenčné čítanie vstupných matíc (máp sveta) a konfigurácií modelov, ako aj na ukladanie výsledných denných štatistík (SimulationDayStats) do textových reportov pre ďalšiu analýzu.

Vývojové nástroje a IDE:

Ako hlavné integrované vývojové prostredie (IDE) bol použitý JetBrains Rider. Toto prostredie bolo zvolené pre jeho pokročilú a rýchlu podporu najnovších štandardov platformy .NET 10 a jazyka C# 14. Rider poskytol integrované nástroje pre hĺbkovú statickú analýzu kódu, inteligentné dopĺňanie a refaktorizáciu. Výkonný debugger uľahčil ladenie stavov simulačného enginu, riadenie asynchrónnych prvkov aplikácie a vizuálne overovanie chovania časovača DispatcherTimer vo WPF rozhraní.

3. Používateľská príručka

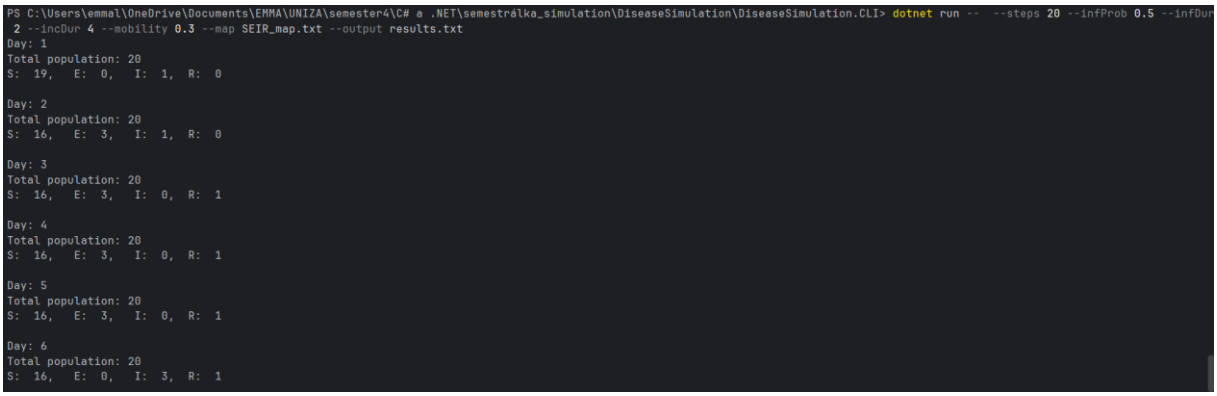
Používateľ ma na výber z dvoch spustiteľných aplikácií – konzolovú a grafickú.

3.1 Konzolová aplikácia

Konzolová aplikácia sa spúšťa z príkazového riadka pomocou príkazu `dotnet run` s argumentami:

ARGUMENT	POPIS	POVINNOSŤ
`--map`	cesta k súbore s mapou s daným simulačným modelom	povinné
`--steps`	počet simulačných krokov	povinné
`--infProb`	pravdepodobnosť nakazenie	povinné
`--infDur`	doba nakazenia	povinné
`--incDur`	doba nakazenia (potrebná pre SEIR model)	povinné
`--mobility`	miera pohyblivosti	povinné
`--output`	cesta k súboru pre zápis štatistických výsledkov	voliteľné
`--help`	návod na použitie a zoznam dostupných príkazov	voliteľné

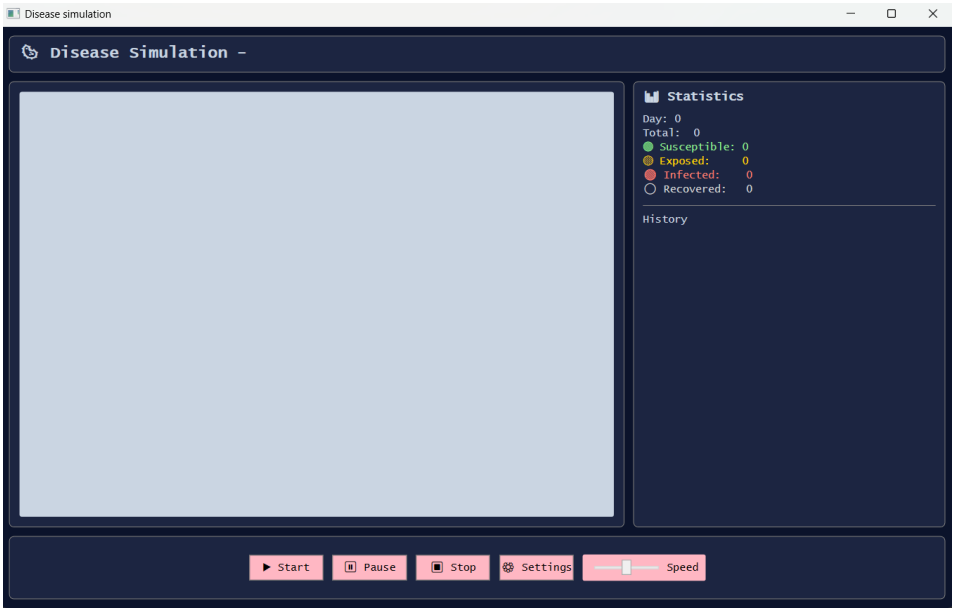
Po spustení aplikácie sa používateľovi vypíše do konzoly denná štatistika vývinu nákazy na základe zadaných argumentov.



Obrázok 4: Ukážka výstupu z konzolovej aplikácie

3.2 Grafická aplikácia

Grafická aplikácia sa spúšťa z príkazového riadku pomocou príkazu `dotnet run`. Po spustení aplikácie sa používateľovi zobrazí hlavné okno s nadpisom a prázdnu plochou pre mapu a vynulovanou štatistikou.

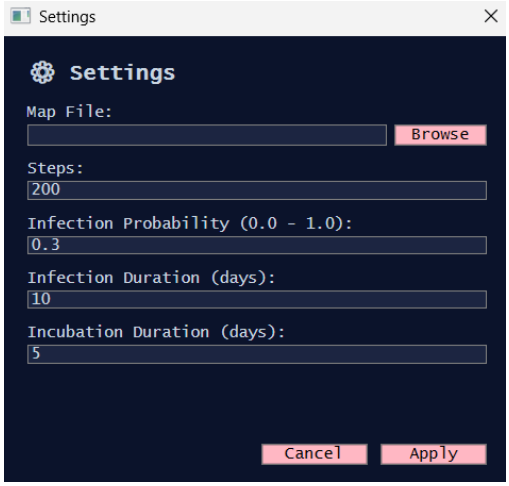


Obrázok 5: Prvotné zobrazenie grafickej aplikácie

Prvým krokom je kliknutie na tlačidlo **Settings**, ktoré otvorí dialógové okno pre výber súboru s mapou vo formáte .txt a nastavenie parametrov simulácie. Aplikácia ponúka 3 typy simulačných modelov:

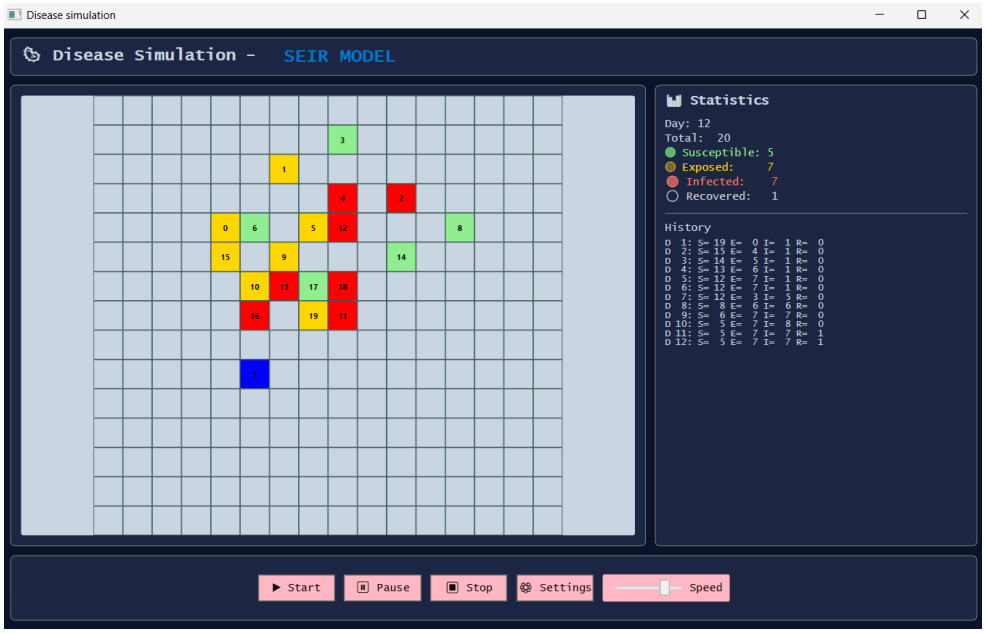
- SIR – zdravý človek sa nakazí, prejde chorobou a potom získa imunitu
- SIS – zdravý človek sa nakazí, prejde chorobou ale imunitu nezískava
- SEIR – zdravý človek príde do kontaktu s nákazou a prejde do inkubácie, po jej skončení sa stane plne infikovaným a nákazlivým, a nakoniec sa uzdraví a získa imunitu

Používateľ ďalej zadáva počet krokov reprezentujúcich dni simulácie, pravdepodobnosť nákazy ako desatinné číslo v rozmedzí 0,0 až 1,0, dĺžku infekcie v dňoch a prípadne dĺžku inkubácie pre SEIR model.



Obrázok 6: Dialógové okna po stlačení tlačidla Settings

Po potvrdení sa mapa načíta a zobrazí sa počiatočný stav populácie. Tlačidlom **Start** sa spustí animovaná simulácia, pričom rýchlosť prehrávania je možné upraviť posuvníkom **Speed** v spodnom paneli. Simuláciu je možné kedykoľvek pozastaviť tlačidlom **Pause** alebo úplne zastaviť a resetovať tlačidlom **Stop**, ktoré vymaže históriu a vráti simuláciu do počiatočného stavu. Počas simulácie sa v pravom paneli zobrazujú aktuálne štatistiky a história všetkých dní.



Obrázok 7: Ukážka funkcie aplikácie pre SEIR modelu

4. Prehľad problémov

Pri tvorbe tohto simulátora som musela vyriešiť niekoľko hlavných problémov, ktoré sa týkali logiky šírenia chorôb a technického spracovania v C#.

Prvou výzvou bolo naprogramovať správanie jednotlivých modelov (SIR, SIS, SEIR). Každý funguje inak: pri SIR bolo treba strážiť trvalú imunitu, pri SIS to, že človek sa môže nakaziť hneď znova, a pri SEIR pribudla inkubačná doba. Pre SEIR som musela každej entite pridať vlastné počítadlo dní, aby systém vedel, kedy sa z nakazeného, ale neinfekčného človeka (Exposed) stane plne infekčný (Infected).

Ďalší problém bol spojený s mriežkou sveta. Nákaza sa šíri lokálne, takže program v každom kroku kontroluje 8 susedných buniek okolo entity. Musela som si dať pozor na ošetrenie okrajov mapy, aby aplikácia nepadala na chybách s indexom poľa. Samotný prenos choroby navyše nie je istý. Prebieha náhodne na základe porovnávania generovaného čísla s nastavenou pravdepodobnosťou (InfectionProbability).

Veľkým orieškom bol aj pohyb ľudí (MobilityRate). Ak by sa entity nehýbali, choroba by zostala na jednom mieste a rýchlo zanikla. Pri pohybe ale vznikali kolízie (viac ľudí chcelo ísť na to isté miesto), čo som vyriešila tak, že si entity pozície jednoducho vymenia (swapnú). Tiež som musela poradie entít pred každým krokom zamiešať, aby neboli zvýhodnení tí jedinci, ktorí sú v pamäti uložení ako prví.

Posledná vec bola grafika vo WPF. Prekresľovanie veľkej mapy a počítanie štatistík (SimulationDayStats) v reálnom čase dalo procesoru zabrať. Keby som celý cyklus spustila len tak v hlavnom vlákne, okno aplikácie by okamžite zamrzlo. Použila som preto DispatcherTimer, ktorý spúšťa simulačné kroky postupne. Vďaka tomu sa mapa plynule prekresľuje a aplikácia normálne reaguje na klikanie v MainWindow aj SettingsWindow.

5. Záver

5.1 Zhodnotenie implementácie

V rámci tejto práce sa mi úspešne podarilo navrhnúť a implementovať funkčný simulátor šírenia chorôb. Podarilo sa mi prepojiť výpočtové jadro v .NET 10 s grafickým rozhraním vo WPF a vytvoriť aj CLI verziu pre rýchlejšie spúšťanie cez príkazový riadok.

Simulácia správne reaguje na zmeny parametrov, ako je pravdepodobnosť nákazy, inkubačná doba či mobilita entít. Výstupy v podobe priebežných grafov a tabuliek štatistík (SimulationDayStats) dávajú jasný prehľad o tom, ako sa epidémia v čase vyvíja pri modeloch SIR, SIS aj SEIR.

5.2 Návrh na ďalší rozvoj

Aj keď je aplikácia plne funkčná, v budúcnosti by sa dala vylepšiť v niekoľkých smeroch:

- **Optimalizácia pre obrovské mapy:** Pri extrémne veľkých mriežkach by sa dal výpočet kroku rozdeliť medzi viacero jadier procesora (paralelné spracovanie pomocou PLINQ alebo Parallel.For), čo by simuláciu ešte viac zrýchlilo.
- **Pokročilejšia grafika:** Klasické WPF prvky by sa dali nahradiť vykresľovaním cez zjednodušené 2D herné knižnice, prípadne by bolo dobré pridať export výsledného grafu priamo do obrázka (PNG/JPG) alebo CSV súboru pre Excel.
- **Komplexnejšie pravidlá:** Do simulačných modelov by sa dali pridať nové faktory, napríklad zavedenie očkovania (vakcinácie), karanténnych zón, kde sa zníži mobilita, alebo rôzny vek entít, kedy by starší jedinci mali dlhšiu dobu liečby.